

 Państwowa Akademia Nauk Stosowanych <i>w Króćnia</i>		Kierunek Informatyka	
Instrukcja do ćwiczćń laboratoryjnych nr:	2	Nazwa przedmiotu: Programowanie w języku Kotlin	
Temat: Podstawowe elementy języka Kotlin - funkcje		Tryb studiów: stacjonarne	
		Czas trwanie ćw. 2x45 min	
Autor materiałów: dr Marcin Skuba			

1. Treści programowe:

Podstawowe elementy języka Kotlin: deklaracja i wywoływanie funkcji, funkcje lambda

2. Cel zajęć:

Celem zajęć jest zrozumienie zasad podstawowych elementów języka Kotlin w porównaniu ze znanymi już takimi jak Java czy C++.

3. Materiały dydaktyczne

I. Deklaracja funkcji

Funkcje w Kotlinie są niezwykle elastyczne. Można je deklarować na wiele sposobów: od tradycyjnych bloków, przez funkcje jednowierszowe, aż po zaawansowane funkcje anonimowe i lambda.

1. Funkcje nie zwracające wartości (Unit) oraz nie pobierające żadnych argumentów

Jeśli funkcja nic nie zwraca (odpowiednik void w Javie), jej typem zwracany jest Unit. Możesz go dopisać jawnie lub pominąć.

```
fun drukujInformacje(): Unit {
    println("Kotlin jest fantastyczny")
}

// Wersja uproszczona (najczęściej stosowana)
fun drukujInformacjeProsto(){
    println("Kotlin jest fantastyczny")
}
```

2. Funkcje pobierające wartości jako argumenty oraz mechanizm przeciążania

Przykład funkcji pobierającej wartości oraz funkcje przeciążone. Mechanizm znany z innych języków programowania np. Javy.

```
//funkcja pobierająca wartość String jako argument
fun pokazMojTekst(text: String){
    for(i in 1..1)
        println("To jest mój tekst: $text")
}

//funkcja przeciążona przez ilość argumentów (String oraz Int)
fun pokazMojTekst(text: String, licznik: Int){
    for(i in 1..licznik)
        println("To jest mój tekst: $text")
}

fun main(){
    pokazMojTekst("Pojedynczy tekst") //wywołanie z jednym argumentem
```

```
pokazMojTekst("Kolejna linia tekstu", 3) //wywołanie z dwoma argumentami
```

3. Funkcje zwracająca wartość

To najbardziej klasyczny sposób, przypominający inne języki, ale z typem zwracany na końcu.

```
fun powitanie(imie: String): String {  
    return "Witaj, $imie!"  
}
```

4. Funkcje jednowyrażeniowe (Single-Expression)

Jeśli funkcja zawiera tylko jedno wyrażenie, możesz pominąć klamry {} oraz słowo return. Kompilator sam domyśli się typu zwracanego.

```
fun pomnoz(a: Int, b: Int) = a * b
```

```
fun getResult(a: Int, b: Int) = if (a != b) 0 else 1
```

5. Parametry domyślne i argumenty nazwane

Kotlin pozwala definiować wartości domyślne, co eliminuje potrzebę przeciążania funkcji (overloading).

```
fun stworzProfil(imie: String="Brak", status: String = "Aktywny", punkty: Int = 0) {  
    println("$imie ($status) - $punkty pkt")  
}
```

```
fun main() {  
    stworzProfil("Adam") // Użyje domyślnych: "Aktywny", 0  
    stworzProfil("Ewa", punkty = 100) // Argument nazwany - pomijamy 'status'  
    stworzProfil(punkty = 220) //Wskazanie nazwy zmiennej punkty z pominięciem  
    pozostałych  
    stworzProfil(punkty = 220, status= "Nieaktywny") //Wskazanie zmiennych w innej  
    kolejności niż w deklaracji  
}
```

6. Funkcje z różną liczbą argumentów (vararg)

Jeśli nie wiesz, ile argumentów przekaże użytkownik, użyj **vararg**.

```
fun sumaWszystkich(vararg liczby: Int): Int {  
    return liczby.sum()  
}
```

```
// Wywołanie: sumaWszystkich(1, 2, 5, 10)
```

7. Funkcje rozszerzające (Extension Functions)

To jedna z najpotężniejszych cech Kotlin. Pozwala "dodać" nową funkcję do istniejącej klasy bez jej modyfikowania.

```
// Dodajemy metodę 'czyParzysta' do klasy Int  
fun Int.czyParzysta(): Boolean {  
    return this % 2 == 0  
}
```

```
val liczba = 10  
println(liczba.czyParzysta()) // true
```

8. Funkcje Lambda i Anonimowe

Lambda to anonimowa funkcja, czyli funkcja bez nazwy, którą możesz przypisać do zmiennej lub przekazać jako argument do innej funkcji.

W Kotlinie wygląda tak:

```
{ parametry -> ciało }
```

- **{ }** – oznacza lambdę
- **parametry** – lista parametrów (może być pusta)
- **->** – separator między parametrami a ciałem
- **ciało** - instrukcje, które wykona lambda

```
val greet = { name: String -> println("Cześć, $name!") }
```

```
greet("Adam")    // wypisze: Cześć, Adam!
```

- greet jest zmienną typu lambda (String) -> Unit
- przekazujesz parametr name
- wykonujesz kod println(...)

- Lambda bez parametrów:

```
val sayHello = { println("Hello world!") }
```

```
sayHello()    // wypisze: Hello world!
```

- Brak parametrów - puste {}
- Można wywołać jak normalną funkcję

- Skróty w lambdach

Jeśli lambda ma **jeden parametr**, możesz użyć it:

```
val result2 = operateOnNumber(7) { it + 3 }    // it = parametr
```

```
println(result2)    // wypisze: 10
```

it automatycznie wskazuje pierwszy parametr

```
val numbers = listOf(1, 2, 3)
```

```
numbers.forEach { println(it) } // 'it' zastępuje każdy element listy
```

- Lambdy z wieloma parametrami

```
val sum: (Int, Int) -> Int = { a, b -> a + b }
```

```
println(sum(3, 4))    // 7
```

- Typ (Int, Int) -> Int oznacza funkcję przyjmującą dwa Inty i zwracającą Int
- Lambda { a, b -> a + b } implementuje ten typ

9. Funkcje wyższego rzędu (Higher-Order Functions)

Funkcja, która przyjmuje inną funkcję jako parametr lub ją zwraca.

```
fun wykonajOperacje(a: Int, b: Int, operacja: (Int, Int) -> Int): Int {
    return operacja(a, b)
}

val wynik = wykonajOperacje(5, 5) { x, y -> x + y } // wynik = 10
```

Najczęściej lambdy używa się, żeby przekazać **kod do wykonania**:

```
fun operateOnNumber(x: Int, operation: (Int) -> Int): Int {
    return operation(x)
}

// użycie
val result = operateOnNumber(5) { number -> number * 2 }

println(result) // wypisze: 10
```

- operation: (Int) -> Int - typ funkcji przyjmującej Int i zwracającej Int
- { number -> number * 2 } - lambda jako argument
- Funkcja operateOnNumber wywołuje ją z x = 5

Typ funkcji	Charakterystyka
Standardowa	Pełna kontrola, wiele linii kodu.
Jednowierszowa	Krótką, zwięzłą, bez return.
Z Unit	Nie zwraca danych (tylko efekt uboczny, np. druk).
Z parametrami domyślnymi	Redukuje liczbę potrzebnych funkcji.
Rozszerzająca	"Dopisuje" logikę do gotowych klas (np. String, Int).
Lambda	Funkcja w formie "skrótów myślowego", idealna do list (np. filter, map).

9. Jawna deklaracja typu funkcyjnego

Jawna deklaracja typu funkcyjnego to sytuacja, w której nie pozwalamy Kotlinowi „zgadywać”, czym jest dana zmienna, ale sami precyzyjnie opisujemy jej „sygnaturę” (czyli co przyjmuje i co zwraca).

Najlepiej zrozumieć to na przykładzie **systemu walidacji hasła**.

Przykład: Walidator Hasła

Wyobraź sobie, że chcesz stworzyć zmienną, która przechowuje regułę sprawdzania hasła. Każda reguła przyjmuje String i zwraca Boolean (czy hasło jest poprawne).

```
// 1. Jawna deklaracja typu: (String) -> Boolean
val isPasswordLongEnough: (String) -> Boolean = { password ->
    password.length >= 8
}
```

```
val hasAdminPrivileges: (Int) -> Boolean = { id ->
    id == 1
}
```

Gdy widzisz dwukropek w deklaracji typu funkcyjnego, czytaj go w ten sposób:

1. **val nazwa:** – „Tworzę stałą o nazwie nazwa...”
2. **(String)** – „...która przyjmuje jako wejście jeden tekst...”
3. **-> Boolean** – „...i po przetworzeniu go, zawsze wyrzuca wartość prawda/fałsz.”
4. **= { ... }** – „...a oto konkretny przepis, jak ma to robić.”

Użycie:

```
fun main() {
    // Użycie isPasswordLongEnough
    val pass1 = "123"
    val pass2 = "tajneHaslo123"

    println(isPasswordLongEnough(pass1)) // Wypisze: false
    println(isPasswordLongEnough(pass2)) // Wypisze: true

    // Użycie hasAdminPrivileges
    val userId = 1
    if (hasAdminPrivileges(userId)) {
        println("Witaj Administratorze!")
    } else {
        println("Brak uprawnień.")
    }
}
```

10. Różnica między jawną deklaracją typu funkcyjnego a zwykłą lambda

- **Zwykła lambda (Inferencja typów)**

W tym przypadku pozwalasz kompilatorowi Kotlin, aby sam wywnioskował typy na podstawie tego, co wpisałeś wewnątrz klamerek {}.

```
// Kotlin sam zgaduje: "Aha, nazwa to String, a wynik to Boolean"
```

```
val isLong = { name: String -> name.length > 5 }
```

Zalety: Pisziesz mniej kodu. Jest to czytelne przy krótkich lambda.

Wady: Jeśli lambda jest długa i skomplikowana, kompilator może mieć problem z "odgadnięciem" typu, albo Ty jako programista stracisz jasny podgląd na to, co ta funkcja ma zwracać.

- **Jawna deklaracja typu funkcyjnego**

Tutaj narzucasz typ **przed** znakiem równości. Mówisz kompilatorowi: "Zmienna validator ma być typu (String) -> Boolean i nie interesuje mnie, jak to sprawdzisz, ale musisz się tego trzymać".

```
// Jawny kontrakt: (String) -> Boolean
```

```
val validator: (String) -> Boolean = { it.length > 5 }
```

Zalety: Masz pełną kontrolę. Jeśli wewnątrz klamerek napiszesz coś, co zwraca Int zamiast Boolean, kompilator natychmiast wyrzuci błąd.

Wartość: Dzięki temu, że typ jest już znany (po lewej stronie), wewnątrz lambdy możesz użyć magicznego słowa `it` (zamiast `name: String`), co jeszcze bardziej skraca kod.

11. Zwracanie lambdy przez zwykłą funkcję

```
fun getPasswordValidator(): (String) -> Boolean {
    return { password -> password.length >= 8 }
}

fun getAdminChecker(): (Int) -> Boolean {
    return { id -> id == 1 }
}

// Użycie:
val myValidator = getPasswordValidator()
println(myValidator("haslo123"))
```

Zadania

Zadanie 1: Podstawy deklaracji i typy zwracane

Zaimplementuj system obliczeń geometrycznych. Stwórz następujące funkcje:

- 1. `calculateRectangleArea` – pobiera długość boków, zwraca pole prostokąta.
- 2. `isTriangleValid` – sprawdza, czy z odcinków o podanych długościach można zbudować trójkąt.
- 3. `convertCelsiusToFahrenheit` – przelicza temperaturę (jak w nazwie funkcji).

Zadanie 2: Przeciążanie funkcji (Overloading)

Stwórz zestaw funkcji o nazwie `formatData`, które będą obsługiwać różne typy wejściowe:

- 1. Przyjmuje `String` i zwraca go w cudzysłowie (np. `"tekst"` -> `" "tekst" "`).
- 2. Przyjmuje `Int` i zwraca informację o jego parzystości (np. `4` -> `"Liczba 4 jest parzysta"`).
- 3. Przyjmuje `List<String>` i zwraca jeden ciąg znaków połączony przecinkami.

Zadanie 3: Parametry domyślne i argumenty nazwane

Zaprojektuj funkcję `generateProfile`, która tworzy opis użytkownika:

- 1. Parametry: `firstName`, `lastName`, `title` (domyślnie `"Mr/Ms"`), `country` (domyślnie `"Poland"`).
 - 2. Stwórz drugą funkcję `createButton`, która przyjmuje `label`, `color` (domyślnie `"Gray"`) oraz `isClickable` (domyślnie `true`).
- W main() wywołaj te funkcje przynajmniej 3 razy, za każdym razem pomijając inne parametry domyślne.*

Zadanie 4: Funkcje jednowyrażeniowe (Single-expression functions)

Zapisz poniższe operacje w formie skróconej (z użyciem znaku `=`):

- 1. `square(n: Int)` – zwraca kwadrat liczby.
- 2. `getMax(a: Int, b: Int)` – zwraca większą z dwóch liczb (użyj `if` jako wyrażenia).
- 3. `isAdult(age: Int)` – zwraca `true`, jeśli wiek wynosi co najmniej 18 lat.

Zadanie 5: Zmienna liczba argumentów (vararg)

Napisz funkcje statystyczne:

- 1. `averageAll` – zwraca średnią liczb podanych jako argumenty (różna liczba wartości).

2. concatenateAll – łączy wszystkie napisy w jeden, oddzielając je spacją. Napisy są przekazywane w kolejnych wartościach (różna liczba wartości).
3. findMin – zwraca najmniejszą z podanych wartości zmiennoprzecinkowych (różna liczba wartości).

Zadanie 6: Funkcje rozszerzające (Extension functions)

Dodaj nową funkcjonalność do istniejących typów bez dziedziczenia:

1. removeWhitespaces() dla String – usuwa wszystkie spacje z napisu (funkcja replace).
2. isPrime() dla Int – sprawdza, czy liczba jest pierwsza.
3. toPercent() dla Double – zamienia liczbę (np. 0.15) na format procentowy ("15.0%").

Zadanie 7: Podstawowe wyrażenia lambda

Zdefiniuj i wywołaj następujące lambdy:

Lambda bezparametrowa (systemReport):

- Zdefiniuj zmienną przechowującą funkcję, która po wywołaniu wypisuje w konsoli komunikat o aktualnym statusie systemu (np. "System operacyjny jest stabilny").
- Wywołaj ją wewnątrz funkcji main na dwa sposoby: używając operatora () oraz metody .invoke().

Lambda z jednym parametrem (displayGreeting):

- Stwórz lambda, która przyjmuje jeden parametr typu String (imię użytkownika).
- Funkcja powinna zwrócić (lub wypisać) spersonalizowane powitanie, np. "Witaj, [imię], miło Cię widzieć!".
- Zwróć uwagę na poprawną deklarację typu parametru wewnątrz klamerek.

Lambda z wieloma parametrami (errorLogger):

- Zaprojektuj funkcję anonimową, która symuluje zapisywanie błędów do dziennika zdarzeń.
- Lambda musi przyjmować dwa argumenty: Int (kod błędu) oraz String (krótki opis problemu).
- Wynik wywołania powinien sformatować te dane w czytelny ciąg znaków, np. "Krytyczny błąd 404: Nie znaleziono zasobu".

Zadanie 8: Skróty w lambdach i słowo kluczowe it

Napisz funkcję wyższego rzędu modifyString(text: String, transformation: (String) -> String): String. Następnie użyj jej w main() do:

1. Zamiany tekstu na wielkie litery (użyj uppercase()).
2. Odwrócenia tekstu (użyj reversed()).
3. Dodania wykrzyknika na końcu.

Zadanie 9: Zaawansowane typy funkcyjne z wieloma parametrami

Zadeklaruj zmienne przechowujące typy funkcyjne i przypisz do nich odpowiednie lambdy:

1. val mathOp: (Double, Double, String) -> Double – lambda, która przyjmuje dwie liczby i znak operacji ("+", "-", "*", "/), a następnie zwraca wynik.
2. val formatAddress: (String, String, Int) -> String – przyjmuje ulicę, miasto i numer domu, zwracając sformatowany adres w jednej linii.
3. val compareLength: (String, String) -> Boolean – zwraca true, jeśli pierwszy ciąg znaków jest dłuższy od drugiego.